ENGINEERING PROGRAM

INT101/INT102 REPORT

TITLE OF THE INTERNSHIP

By:

- Yasine & Mtibaa
- Mahdi & Hachicha
- Mouhamed & Baccouch
- Hana & Gdoura
- Fares & Smaoui
- Selima & Grissa

Academic Supervisor:

First & Last Name

Institution Supervisor:

First & Last Name

Institution Name

Tunis, 2023 2024

ABSTRACT

The abstract is a concise summary of the main points and findings of your project. It provides a brief overview of the purpose, methodology, results, and conclusions of your research or study. The abstract should be written in a clear and concise manner, using language that is accessible to a broad audience.

Keywords:

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

1 EXECUTIVE SUMMARY

1.1 Project Overview

1.2 Client Brief & Objectives

1.3 Key Milestones & Timeline

1.4 Document Conventions

2 INTRODUCTION

2.1 Background

2.2 Problem Statement

2.3 Scope of Work & Deliverables

2.4 Report Structure

3 COMPANY CONTEXT

3.1 Description of the Company

3.2 Mission and Objectives

3.3 Industry Structure

3.4 Market Structure

4 RESEARCH PHASE

4.1 Market & Competitor Analysis

4.2 Feasibility Study

4.3 Standards & Regulatory Review

4.4 Technology Stack Selection

4.5 Reference Architecture Review

5 SYSTEMS ARCHITECTURE

5.1 System Overview Diagram

5.2 Electrical & Power Systems

5.3 Electronic Control Units (ECUs)

5.4 Embedded Software Architecture

The embedded software running on the ESP32 microcontroller is structured as a **single-threaded, cooperative loop** built on the Arduino framework. This architecture was chosen for its simplicity and determinism: all BMS-related tasks execute sequentially on one core, eliminating the need for inter-task synchronization.

The software is divided into five logical layers, shown in Figure 1:

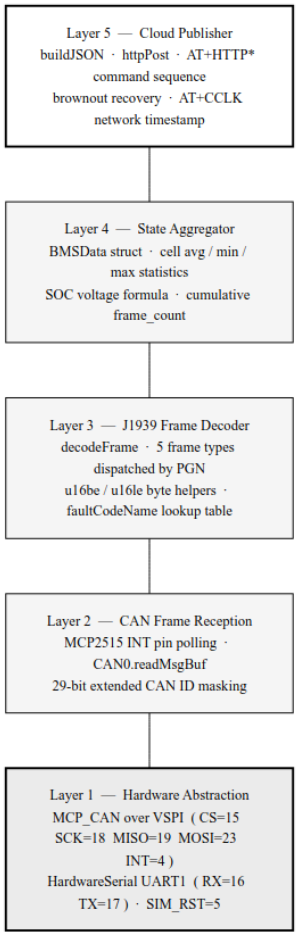

Figure 1: ESP32 embedded software layer diagram

Hardware Abstraction Layer (HAL) Provides direct access to the MCP2515 via SPI (`mcp_can` library) and to the SIM800L via HardwareSerial UART1. Pin assignments and bus parameters (CS, INT, baud rate, crystal frequency) are defined as constants and isolated from the application logic.

CAN Frame Reception The MCP2515 asserts the `INT` line (GPIO 4) when a frame is placed in its receive buffer. The main loop polls this line and calls `CAN0.readMsgBuf()` to drain all pending frames. Extended CAN IDs are masked from the 29-bit field before processing.

J1939 Decoder The `decodeFrame()` function dispatches each received frame by inspecting the `FUNC` byte of the CAN ID (bits 23–16). It handles five frame types (cell voltages, temperatures, BMS Basic 1 & 2, charging request) and writes decoded values into the `BMSData` struct.

State Aggregator The `BMSData` struct accumulates all decoded fields and maintains derived statistics (cell average, min, max, voltage-based SOC). The struct is updated in place and always holds the most recent complete state of the battery.

Cloud Publisher Every 5 seconds, `buildJSON()` serializes `BMSData` into a JSON document using the ArduinoJson library, and `httpPost()` delivers it to the VPS via the SIM800L AT+HTTP* command set.

5.4.1 Key Design Decisions

No RTOS. A FreeRTOS task structure was considered but rejected because the single-responsibility loop handles all timing requirements naturally: CAN frames arrive at 100–500 ms intervals, and the publish cycle runs every 5 seconds. Adding preemptive scheduling would have introduced stack management and mutex overhead without any functional benefit.

CAN drain during HTTP wait. The SIM800L's `AT+HTTPACTION` response can take up to 30 seconds over slow GPRS. To prevent the MCP2515 receive buffer (6 frames deep) from overflowing during this window, the HTTP wait loop polls the CAN `INT` pin and drains frames continuously, calling `decodeFrame()` in-place. This ensures the `BMSData` struct is fully up-to-date when the next publish cycle begins.

HardwareSerial over SoftwareSerial. SoftwareSerial operates via bit-banging and is sensitive to interrupt latency. The SIM800L produces multi-line AT response bursts that SoftwareSerial occasionally drops bytes from at 9600 baud on a loaded core. HardwareSerial UART1 uses dedicated silicon and a hardware FIFO, making it robust under all operating conditions.

5.5 Cloud & Connectivity Architecture

The cloud and connectivity architecture connects the on-vehicle ESP32 to a remote browser with minimal intermediaries. The full pipeline is shown in Figure 2.

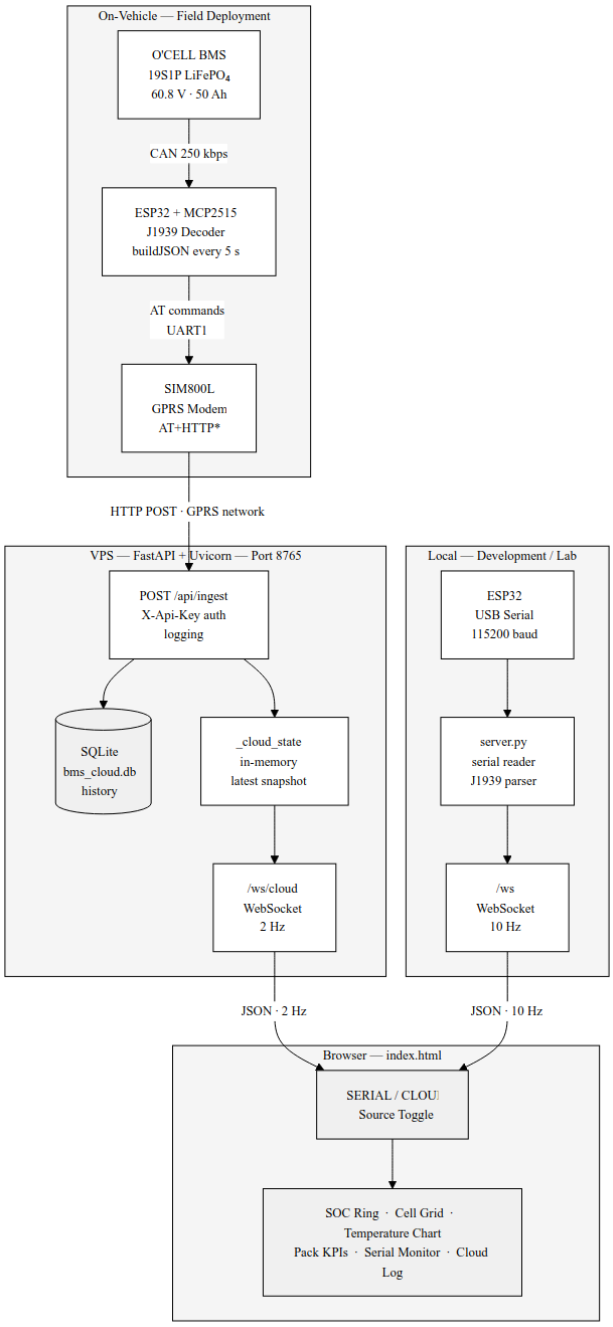

Figure 2: End-to-end cloud connectivity architecture

5.5.1 Component Roles

ESP32 + MCP2515 (On-vehicle edge node) Reads and decodes SAE J1939 CAN frames from the BMS at up to 10 frames per second, aggregates them into a snapshot, and transmits one JSON document to the VPS every 5 seconds.

SIM800L GSM/GPRS Modem Provides wide-area cellular connectivity using the carrier GPRS network. Communicates with the ESP32 via AT commands over UART. Handles TCP/IP, HTTP, and DNS entirely on-chip using the AT+HTTP* service, so the ESP32 only needs to manage the AT command dialogue.

VPS Backend (FastAPI + Uvicorn) Acts as the single ingestion point and real-time relay for all cloud data. Receives POST requests from the ESP32, stores them to SQLite for persistence, and maintains an in-memory state that is pushed to connected browser clients via WebSocket at 2 Hz. The server also handles the SERIAL mode WebSocket, allowing the same dashboard to be used with a direct USB connection.

SQLite Database Provides persistent local storage for all received snapshots. Enables historical data retrieval via the `/api/history` endpoint without any external database service.

Browser Dashboard A single-page web application that renders live battery data received over WebSocket. Supports SERIAL mode (direct USB, 10 Hz) and CLOUD mode (VPS relay, 2 Hz). Auto-reconnects on network interruption.

5.5.2 Data Flow Summary

1. O'CELL BMS → CAN bus (250 kbps, SAE J1939)
2. ESP32 + MCP2515 → decode all frames → `BMSData` struct
3. ESP32 → `buildJSON()` → JSON document
4. ESP32 → SIM800L AT+HTTPACTION=1 → `POST /api/ingest` (HTTP, GPRS)
5. VPS server → validate API key → SQLite insert + in-memory update
6. VPS server → `/ws/cloud` WebSocket push every 2 s
7. Browser → render panels (SOC, cells, temperatures, faults, cloud log)

5.5.3 Dual-Mode Operation

A key architectural feature is that the dashboard supports two independent data sources via the same frontend interface. In **SERIAL mode**, the backend reads raw CAN log lines from the ESP32 USB serial port, decodes them through its own Python J1939 parser (`BMSState.decode()`), and pushes the result at 10 Hz. In **CLOUD mode**, the backend serves the latest snapshot received from the ESP32 over GPRS.

This dual-mode design provides two practical benefits:

- During development and testing, the system can be operated entirely over USB without requiring a SIM card or GPRS connection.
- In the field, the dashboard can be opened from any web browser pointed at the VPS IP address, with no local software to install.

5.6 3D Mechanical Architecture

5.7 Safety & Redundancy Design

6 PREPARATION PHASE

6.1 Project Governance & Team Structure

6.2 Development Environment Setup

6.3 Simulation & Prototyping Environment

6.4 Risk Register (Initial)

6.5 Procurement & Vendor Selection

6.6 Prototype Planning

7 CIRCUIT DESIGN & ELECTRONICS

7.1 Schematics & PCB Design

7.2 Battery Management System (BMS) Design

7.3 Motor Drive & Inverter Design

7.4 Charging System Design

7.5 Sensor & Actuator Interfacing

7.6 Design Review & Simulation Results

8 3D DESIGN & MECHANICAL ENGINEERING

8.1 Chassis & Body Design

8.2 Battery Pack Housing & Thermal Management

8.3 Powertrain Packaging

8.4 Electrical Harness Routing

8.5 Design for Manufacture (DFM) Review

8.6 3D Render & Visualization Deliverables

9 SOFTWARE & CODE IMPLEMENTATION

This section describes the software developed for the Bako OBD ISS platform, covering the embedded firmware running on the ESP32 microcontroller, the battery management decoding logic, the cloud publishing pipeline, and the validation strategy used to verify correct behavior at each layer.

9.1 Firmware Architecture & RTOS Integration

The ESP32 firmware is built on the **Arduino framework** using the **PlatformIO** build system, which provides dependency management, cross-compilation, and serial monitoring in a single toolchain. Although the Arduino framework does not expose the underlying FreeRTOS scheduler directly, the ESP32's dual-core architecture is leveraged implicitly: the Arduino `loop()` function executes on Core 1 while the Wi-Fi/Bluetooth stack occupies Core 0. All BMS-related tasks — CAN frame reception, state accumulation, and HTTP publishing — run sequentially on Core 1 without preemption, which simplifies state management and eliminates the need for mutexes at the firmware level.

The firmware follows a **cooperative, interrupt-driven** model. The MCP2515 CAN transceiver asserts an active-low `INT` line (GPIO 4) each time a new frame is placed in its receive buffer. The main loop polls this pin continuously and drains all pending frames before entering the timed publish cycle. This design ensures that no CAN frame is ever dropped due to buffer overflow, even during the longer HTTP transaction window.

The overall execution model is summarized in Figure 3.

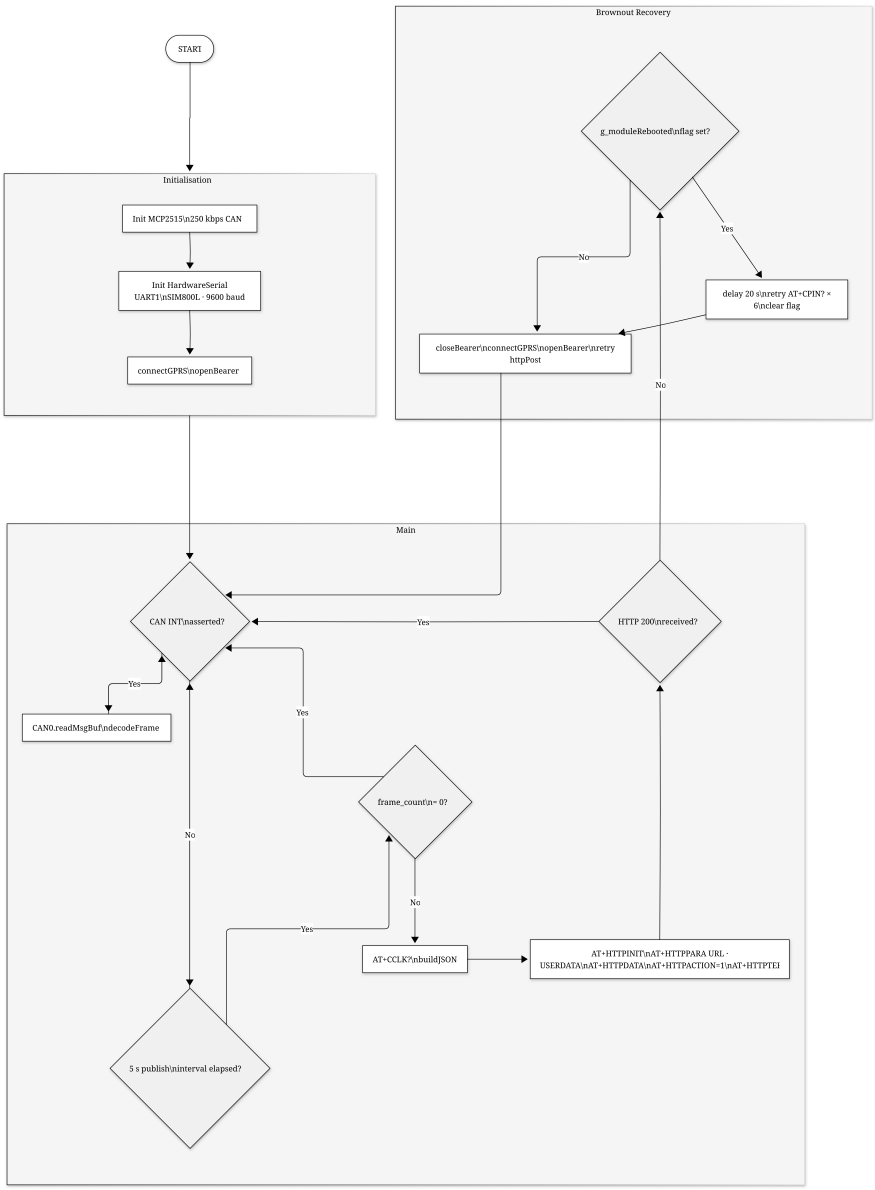

Figure 3: Firmware main-loop execution model: CAN drain → publish every 5 s

The SIM800L GSM modem is driven via **HardwareSerial UART1** (GPIO 16/17) at 9600 baud. HardwareSerial was chosen over SoftwareSerial because the SIM800L's AT+HTTP* responses can arrive as multi-line bursts that SoftwareSerial's byte-at-a-time interrupt handler occasionally corrupts at 9600 baud on a loaded CPU.

9.2 BMS Firmware

The BMS firmware is responsible for reading raw CAN frames from the O'CELL IFS60.8-500-F-E3 battery management system, decoding them according to the **SAE J1939** standard, maintaining

a live BMS state structure, and serializing that state to JSON for cloud upload.

9.2.1 CAN Bus Interface

The physical CAN interface uses a **MCP2515** controller connected to the ESP32 via the VSPI bus at 10 MHz. The bus operates at **250 kbps**, which is the standard J1939 baud rate for vehicle control networks. All frames from the O'CELL BMS use 29-bit extended CAN identifiers, formatted according to the J1939 Parameter Group Number (PGN) convention:

$$\text{CAN ID} = 0x18\_\text{FUNC\_SUB\_SA} \tag{1}$$

where `FUNC` is the PGN function byte that identifies the frame type, `SUB` encodes sub-addressing or destination, and `SA` = 0xF4 is the source address of the BMS module.

The MCP2515 library is configured in `MCP_ANY` mask mode so that all frames on the bus are received without hardware filtering. Software-side filtering is performed in the `decodeFrame()` function by inspecting the `FUNC` and `SUB` bytes extracted from the 29-bit ID.

9.2.2 Supported Frame Types

Five distinct frame types are decoded, covering all BMS data needed for monitoring and fault diagnosis. Table 1 lists each frame with its CAN ID, transmission rate, and data content.

Table 1: O'CELL BMS CAN Frame Map (SAE J1939, 250 kbps)

CAN ID	Name	Rate (ms)	Content
`0x18C8--CC28F4`	Cell voltages (5 frames)	500	4 cells each, big-endian uint16, 1 mV/bit
`0x18B428F4`	Temperature probes	500	Up to 8 probes, raw − 40 = °C
`0x18FF28F4`	BMS Basic Message 1	100	SOC, current, voltage, fault flags
`0x18FE28F4`	BMS Basic Message 2	100	Min/max cell mV, discharge limit
`0x18FFE5F4`	Charging request	500	Max charge V, max charge A, start signal

Cell Voltage Frames (0x18C8–CC28F4). The 19 battery cells are transmitted across five consecutive frames. Each frame carries four cells encoded as big-endian 16-bit unsigned integers, with a resolution of 1 mV per bit. The last frame (0x18CC28F4) covers cells 17–19 and pads the unused fourth slot with 0x0000. The firmware detects and skips zero-padded slots:

```
for (int i = 0; i < 4; i++) {
    uint16_t mv = u16be(data, i * 2);
    if (mv != 0 && (base + i) < 19)
        bms.cell_mv[base + i] = mv;
```

}

After each group update, aggregate statistics (minimum, maximum, average) are recomputed from all non-zero cells, and a voltage-based SOC estimate is derived as described in Section 9.2.4.

BMS Basic Message 1 (0x18FF28F4). This 100 ms frame carries the core operational state of the pack. The byte layout is as follows:

- **Byte 0** — Status bit-field: charge cable connected (bit 0), charging in progress (bit 1), discharging in progress (bit 2), BMS ready (bit 3), discharge contactor closed (bit 4), charge contactor closed (bit 5).
- **Byte 1** — State of charge from the BMS coulomb counter (0–100%).
- **Bytes 2–3** — Pack current as a little-endian uint16 with a -5000 offset and 0.1 A/bit scaling. Positive values indicate discharge; negative values indicate charging.
- **Bytes 4–5** — Direct BMS pack voltage, little-endian uint16, 0.1 V/bit.
- **Byte 6** — Fault severity level (0 = normal, 1 = serious fault).
- **Byte 7** — Fault code (see Table 2).

Table 2: BMS Fault Code Map (byte 7 of 0x18FF28F4)

Code	Description
0x00	No fault (normal operation)
0x01	Over-temperature — severe
0x02	Total pack voltage too high
0x03	Total pack voltage too low
0x04	Discharge over-current
0x05	Cell voltage too high
0x06	Cell voltage too low

Temperature Probes (0x18B428F4). Up to eight thermal probes are encoded as single bytes using the formula $T[°\mathrm{C}] = \mathrm{raw} - 40$. A raw value of 0xFF indicates a disconnected sensor and is skipped. The O'CELL pack fitted to the vehicle contains four active probes.

Charging Request (0x18FFE5F4). This frame is transmitted by the BMS toward the on-board charger. It specifies the maximum allowable terminal voltage (bytes 0–1, 0.1 V/bit) and maximum charge current (bytes 2–3, 0.1 A/bit). Bit 0 of byte 4 being clear signals that the BMS is requesting the charger to begin charging.

9.2.3 BMS State Structure

All decoded values are accumulated into a single `BMSData` struct that persists across frames and represents the most recent complete snapshot of the battery's state. The struct is updated in-place by `decodeFrame()` and read atomically by `buildJSON()` at each publish cycle. Because the firmware runs single-threaded, no locking is required.

9.2.4 SOC Calibration

The firmware uses a **voltage-based SOC estimate** derived from the average cell voltage. This method was selected because it correlates closely with the vehicle's own dashboard reading, unlike the BMS internal coulomb counter which can drift over time.

The calibration was performed over four separate on-vehicle sessions, comparing the computed average cell voltage with the vehicle's onboard display:

$$\text{SOC}[\%] = \frac{\overline{V}_{\text{cell}} - 2500}{3387 - 2500} \times 100 \tag{2}$$

where $\overline{V}_{\text{cell}}$ is the average cell voltage in millivolts. The lower bound of 2500 mV/cell corresponds to 0% SOC (47.50 V pack), and 3387 mV/cell was calibrated to match 100% on the car display (64.35 V pack). Values above 3387 mV, which occur during active charging, are clamped to 100%.

Table 3 shows the four calibration sessions used to arrive at the 3387 mV reference point.

Table 3: SOC Calibration Data — Four On-Vehicle Sessions

Session	Avg cell (mV)	Car display (%)	Solved top (mV)
2026-03-31 10:06	3301.6	89.0	3400.7
2026-03-31 10:57	3306.2	89.0	3405.8
2026-04-01 13:03	3284.8	89.4	3377.9
2026-04-01 16:04	3285.0	88.4	3387.4
		Adopted reference	**3387 mV**

9.2.5 Cloud Publishing via SIM800L

At every 5-second publish interval, the firmware calls `buildJSON()` to serialize the current `BMSData` struct into a JSON string, then calls `httpPost()` to deliver it to the VPS backend via SIM800L GPRS.

The HTTP transaction uses the SIM800L's native **AT+HTTP*** command set over plain HTTP (AT+HTTPSSL=0). The sequence is:

1. `AT+HTTPTERM` — close any stale session from a previous transaction.
2. `AT+HTTPINIT` — open a new HTTP session.
3. `AT+HTTPPARA="URL",...` — set the full endpoint URL.
4. `AT+HTTPPARA="USERDATA","X-Api-Key: `*key*`\r\n"` — inject the API authentication header.
5. `AT+HTTPDATA=`*len*`,10000` — upload the JSON body.
6. `AT+HTTPACTION=1` — execute the HTTP POST and wait for `+HTTPACTION:` response.
7. `AT+HTTPTERM` — close the session.

During the AT+HTTPACTION wait loop (up to 30 s), the firmware continues draining CAN frames from the MCP2515 receive buffer so that no BMS data is lost while the modem is busy.

9.2.6 Brownout Recovery

A known failure mode of the SIM800L is a mid-transaction power brownout caused by the $\sim$1.5 A peak current during GPRS transmission exceeding the supply rail's transient capability. When this occurs, the modem resets and emits an unsolicited `"Call Ready"` string in its UART output.

The firmware detects this string in every AT response buffer (both `sendAT()` and `sendATExpect()` and the `HTTPACTION` wait loop) and sets a global `g_moduleRebooted` flag. When a publish cycle fails and this flag is set, the recovery sequence is:

1. Wait 20 s for the SIM card and GSM stack to re-initialize.
2. Retry `AT+CPIN?` up to six times at 4-second intervals.
3. Re-open the GPRS bearer and retry the publish once more.

The long-term hardware fix is to add a 1000 µF / 10 V bulk capacitor close to the SIM800L VCC pin.

9.3 Motor Control Algorithms

9.4 Vehicle Management Unit (VMU) Software

9.5 OTA Update & Secure Boot

9.6 Unit & Integration Testing

Software testing for the BMS firmware and cloud pipeline was conducted at three levels: unit testing of the decoding logic, integration testing of the full CAN-to-cloud pipeline, and func-

tional testing of the live dashboard.

9.6.1 Unit Testing — Frame Decoder

Each frame type in `decodeFrame()` was verified by replaying known raw bytes and checking the decoded output against hand-computed expected values. The test case below illustrates verification of cell voltage frame 0x18CB28F4:

```
Frame ID: 0x18CB28F4   Data: 0D 3F  0D 38  0D 3A  0D 39


0x0D3F = 3391 mV  -> Cell 13   (big-endian, group = 0xCB - 0xC8 = 3)
0x0D38 = 3384 mV  -> Cell 14
0x0D3A = 3386 mV  -> Cell 15
0x0D39 = 3385 mV  -> Cell 16
```

Similarly, the signed current decoding formula (offset −5000, scale 0.1 A/bit) and the SOC formula (Equation (2)) were verified against values from the real car log file `bms_log_2026-03-10T10-20-02.txt`, which contains 3197 frames captured during an on-vehicle session.

9.6.2 Integration Testing — Cloud Pipeline

The end-to-end pipeline was tested without live hardware using the `replay_to_cloud.py` tool, which parses the captured CAN log and replays decoded JSON snapshots to `POST /api/ingest` at configurable speed. This allowed the following verifications:

- The `/api/ingest` endpoint correctly rejects requests with an invalid `X-Api-Key` header (HTTP 401).
- Each accepted snapshot is stored in the SQLite `bms_cloud.db` database with correct timestamp and device ID.
- The `/ws/cloud` WebSocket pushes the most recent snapshot to all connected browser clients within 2 seconds of ingestion.
- The `/api/history` endpoint returns records newest-first with correct JSON structure.

9.6.3 Functional Testing — Live Dashboard

The live dashboard (`frontend/index.html`) was tested in both SERIAL and CLOUD modes:

SERIAL mode ESP32 connected via USB to a development laptop. The server decoded incoming frames in real time and the 19-cell voltage grid, SOC ring, and temperature bars updated at 10 Hz without visible lag.

CLOUD mode Data replayed from the log file through `/api/ingest`. The dashboard received snapshots via `/ws/cloud` at 2 Hz and all panels rendered correctly, including the cloud communication log panel which displayed timestamped RECV and WS events.

Reconnect behavior The backend was deliberately stopped while the dashboard was open. The browser WebSocket closed, the connection status indicator turned red (Disconnected), and the overlay appeared. On server restart, the client automatically reconnected within 5 seconds and resumed live updates.

10 CLOUD CONNECTIVITY & IOT

This section describes the complete cloud telemetry pipeline designed and implemented for the Bako OBD ISS platform. The pipeline carries live battery data from the vehicle's CAN bus to a remote web dashboard through a sequence of embedded, network, server, and browser components. The architecture was designed to be hardware-independent: the same dashboard and backend serve both a direct USB serial connection (for lab use) and a cellular GPRS connection from the field.

10.1 Telematics & Data Acquisition

Telematics in this project refers to the capture, local decoding, and wireless transmission of battery state data from the moving vehicle to a remote server.

10.1.1 On-Vehicle Data Acquisition

Data acquisition begins at the O'CELL IFS60.8-500-F-E3 BMS, which continuously transmits CAN frames on a 250 kbps SAE J1939 bus. The ESP32 microcontroller, equipped with an MCP2515 CAN transceiver, receives all frames on this bus and decodes them in real time using the firmware described in Section 9.2.

The BMS transmits at two rates. High-priority operational frames (0x18FF28F4 and 0x18FE28F4) arrive every 100 ms and carry pack voltage, current, SOC, and fault status. Lower-priority frames carrying cell-level voltages and temperatures arrive every 500 ms. At 250 kbps the bus is far from saturated; the five BMS frame types together occupy less than 2% of available bandwidth.

10.1.2 Snapshot Aggregation and Publish Cadence

Because the SIM800L HTTP transaction takes up to 5 seconds over GPRS (bearer setup, DNS, TCP handshake, data transfer), it is not feasible to transmit every individual CAN frame to the cloud. Instead, the firmware implements a **snapshot model**: all incoming frames are decoded and merged into a single `BMSData` struct that always holds the most recent value for each field. Every 5 seconds, this struct is serialized into a single JSON document and sent as one HTTP POST to the VPS.

This approach has several advantages:

- The cloud endpoint receives at most 12 snapshots per minute regardless of CAN bus activity.
- Each snapshot is self-contained and requires no state from previous snapshots to be inter-

preted.

- CAN frames continue to be decoded during the HTTP transaction, so the struct is fully populated before the next publish cycle.

10.1.3 Cellular Connectivity — SIM800L GPRS

Wireless transmission is handled by a **SIM800L GSM/GPRS modem** connected to the ESP32 via HardwareSerial UART1. The modem establishes a GPRS bearer using the carrier's APN, then uses the AT+HTTP* command set to perform a standard HTTP POST to the VPS endpoint.

The connection lifecycle at startup is:

1. `AT+CPIN?` — verify SIM card is ready (retried up to 6 times after a reboot).
2. `AT+CSQ` — check signal quality.
3. `AT+CREG?` — confirm network registration.
4. `AT+SAPBR=3,1,...` — configure bearer (APN, username, password).
5. `AT+SAPBR=1,1` — open the GPRS bearer (up to 25 s).
6. `AT+SAPBR=2,1` — confirm the bearer is active and has an IP address.

If the bearer fails to open, the ESP32 retries the full sequence up to three times before invoking a soft restart (`ESP.restart()`).

10.1.4 Network Timestamp

To provide accurate event timestamps regardless of the vehicle's location, the firmware reads the network time from the SIM card using `AT+CCLK?` before each publish cycle. The returned timestamp (format `yy/MM/dd,HH:mm:ss±zz`) is included in the JSON payload as the `timestamp` field. If the modem does not return a valid timestamp (e.g., during initial bearer setup), the firmware falls back to `uptime_ms` from `millis()`.

10.2 Cloud Platform Configuration

10.2.1 Platform Choice

The cloud backend runs on a **Virtual Private Server (VPS)** — a Linux instance with a public IP address — rather than a managed cloud database service. This design decision was made for the following reasons:

- **No third-party dependency**: the entire data path (ingest → storage → streaming) is contained within a single process under direct control of the project team.
- **Simplicity**: the SIM800L communicates over plain HTTP, eliminating the need for TLS certificate handling on the embedded side.
- **Cost**: a VPS is significantly cheaper than a managed real-time database for low-throughput IoT workloads.
- **Full observability**: the server log, database file, and WebSocket connections are all directly inspectable via SSH.

10.2.2 Backend Runtime

The backend is a single Python file (`backend/server.py`) running under **Uvicorn**, an ASGI server. The application framework is **FastAPI**, which provides automatic OpenAPI documentation, WebSocket support, and asynchronous request handling. The dependencies are minimal by design:

Table 4: Backend Python Dependencies

Package	Version	Purpose
`fastapi`	0.115.0	Web framework, WebSocket, routing
`uvicorn`	0.29.0	ASGI server (serves HTTP and WebSocket on one port)
`pyserial`	3.5	Serial port I/O for USB/SERIAL mode

No external database client, message broker, or cloud SDK is required. The entire stack is started with a single command:

```
python server.py --cloud-only
```

The `--cloud-only` flag suppresses serial port detection so the server starts cleanly on a headless VPS without a USB device attached.

10.2.3 Environment Variables

Server behavior is controlled by three environment variables, summarized in Table 5:

Table 5: Server Environment Variables

Variable	Default	Purpose
`BMS_API_KEY`	`bako-bms-2024`	Shared secret for `POST /api/ingest` authentication
`HOST`	`0.0.0.0`	Bind address (all interfaces on VPS)
`PORT`	8765	TCP port for HTTP and WebSocket traffic

10.3 Data Pipeline & Storage

10.3.1 Pipeline Overview

The end-to-end data pipeline operates as follows and is illustrated in Figure 4:

1. The ESP32 reads CAN frames and decodes them into a `BMSData` struct (on-vehicle).
2. Every 5 seconds, the struct is serialized into a JSON document and sent as `POST /api/ingest` to the VPS, with an `X-Api-Key` header for authentication.
3. The server validates the API key, stores the payload in SQLite, and updates the in-memory `_cloud_state` dictionary.
4. Browser clients connected to `/ws/cloud` receive the latest `_cloud_state` every 2 seconds via WebSocket push.

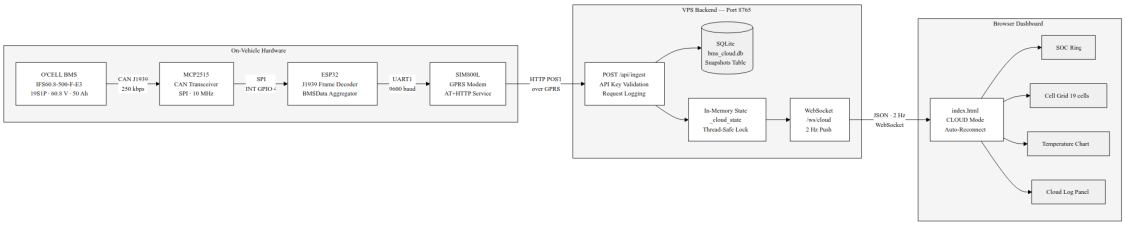

Figure 4: Complete data pipeline from CAN bus to browser

10.3.2 Ingest Endpoint

The `POST /api/ingest` endpoint is the sole entry point for ESP32 data. Its implementation:

1. Validates the `X-Api-Key` header against the `BMS_API_KEY` environment variable. An incorrect key returns HTTP 401.
2. Extracts the timestamp, device ID, cell count, SOC, and pack voltage from the JSON body for logging.

3. Runs a non-blocking SQLite insert via `asyncio.run_in_executor()` to avoid blocking the event loop during disk I/O.
4. Updates the in-memory `_cloud_state` dict atomically under a threading lock (because the serial reader thread and the asyncio event loop share this state).
5. Appends a timestamped `[RECV]` entry to the cloud communication log.
6. Returns `{"status": "ok", "ts": ...}` with HTTP 200.

10.3.3 SQLite Persistent Storage

Received snapshots are persisted in a local SQLite database (`backend/bms_cloud.db`) for historical analysis and post-session review. The schema is intentionally minimal:

```
CREATE TABLE IF NOT EXISTS snapshots (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    ts        TEXT    NOT NULL,
    device_id TEXT    NOT NULL DEFAULT 'esp32',
    payload   TEXT    NOT NULL
);
```

The full JSON payload is stored as a text column, which avoids schema migrations when new fields are added to the firmware. Queries for the `/api/history` endpoint retrieve records ordered by descending `id` (newest first) with a configurable limit. SQLite was chosen over a full database server because:

- At one snapshot per 5 seconds, a full day of operation produces only $\sim$17,000 rows ($\sim$10 MB), well within SQLite's performance envelope.
- No separate database process or network socket is required.
- The database file can be copied directly from the VPS for offline analysis with standard tools.

10.3.4 In-Memory State and WebSocket Push

In addition to SQLite, the server maintains an in-memory dictionary (`_cloud_state`) that always holds the most recently received snapshot. This allows the `/ws/cloud` WebSocket handler to serve browser clients at 2 Hz with zero disk I/O per push.

The in-memory state is initialized to `{"connected": false, "source": "cloud"}` on startup. The dashboard interprets `connected: false` as a "waiting for ESP32" state and displays a yellow indicator to the user rather than an error.

10.4 Remote Monitoring Dashboard

10.4.1 Architecture

The dashboard is a **single-page web application** served as a static HTML file (`frontend/index.html`) directly by the FastAPI backend via `GET /`. No separate web server, build tool, or JavaScript framework is required. All rendering logic is contained in vanilla JavaScript.

The dashboard connects to the backend over **WebSocket**, which provides a persistent, low-latency bidirectional channel. In SERIAL mode the WebSocket endpoint is `/ws` (10 Hz push from the serial reader); in CLOUD mode it is `/ws/cloud` (2 Hz push from the in-memory state). The user switches between modes using a toggle button in the header.

10.4.2 Dashboard Panels

Table 6 summarizes the information panels displayed by the dashboard.

Table 6: Dashboard Panel Summary

Panel	Contents
SOC ring	Animated circular gauge showing voltage-derived SOC. BMS coulomb-counter and secondary SOC values are displayed below.
Pack KPIs	Pack voltage, current, discharge limit, charge request, and cell count in a card grid.
Cell grid	All 19 cells — individual voltage bars color-coded by threshold (overvoltage / full / good / normal / low / undervoltage), with min/max/avg/spread statistics.
Temperature	Up to 4 temperature probe readings with color-coded bars and a rolling time-series chart.
Serial monitor	Raw CAN frame log with text export and Excel export capabilities.
Cloud log	Real-time cloud communication events (RECV / WS / ERR), polled from `/api/cloud-log` every 2 seconds, independent of the SERIAL/CLOUD source toggle.

10.4.3 Connection Status Indicator

The dashboard header contains a three-state connection indicator that provides immediate visual feedback:

Green — Live WebSocket is open and the last received message contained `connected: true`.

Yellow — Waiting WebSocket is open but `connected: false` (backend is running but no ESP32 data has arrived yet).

Red — Disconnected WebSocket is closed or encountered a network error.

10.4.4 Auto-Reconnect

If the WebSocket closes unexpectedly (server restart, network interruption), the browser automatically attempts to reconnect every 5 seconds. The overlay is re-displayed during the disconnected interval. This behavior is handled entirely client-side and requires no intervention from the user.

10.5 Security & Cybersecurity

10.5.1 API Key Authentication

The `POST /api/ingest` endpoint is protected by a shared secret transmitted as the `X-Api-Key` HTTP header. The server compares the received key against the `BMS_API_KEY` environment variable. Requests with a missing or incorrect key are rejected with HTTP 401 before any data is read from the body, and an `[ERR]` entry is appended to the cloud log.

The API key is configured separately on the firmware (`VPS_API_KEY` in `include/secrets.h`, which is in `.gitignore`) and on the server (environment variable), so the secret is never committed to the repository.

10.5.2 Transport Layer

Communication between the SIM800L and the VPS uses plain HTTP. While this means the payload is not encrypted in transit, the following mitigating factors apply to this deployment context:

- The data transmitted is battery telemetry (voltages, temperatures, SOC) — not personally identifiable information or control commands.
- The VPS port is firewalled to accept connections only on the designated port.
- The API key protects against unauthorized data injection.

For a production deployment, upgrading to HTTPS requires either a SIM800L module with firmware $\geq$ R14.18 (`AT+HTTPSSL=1` support) or replacing the modem with a SIM7600/A7670 module that supports native TLS.

10.5.3 WebSocket and Dashboard Access

The dashboard itself is served over plain HTTP and the WebSocket connection is unencrypted. Since the dashboard is intended for use on a private network or VPN-protected VPS, this is considered acceptable for the current project scope. Adding HTTPS would require an SSL certificate (e.g., from Let's Encrypt) and an Nginx reverse proxy, which is a straightforward extension if needed.

10.6 API Documentation

The FastAPI backend exposes the following endpoints, summarized in Table 7. FastAPI generates interactive Swagger UI documentation automatically at `/docs` when the server is running.

Table 7: Backend API Endpoint Reference

Method	Path	Description
GET	`/`	Serves the dashboard HTML file.
WS	`/ws`	Serial stream — pushes BMS JSON to the browser at 10 Hz. Active when an ESP32 is connected via USB.
WS	`/ws/cloud`	Cloud stream — pushes the latest in-memory snapshot at 2 Hz.
POST	`/api/ingest`	Receives a BMS JSON snapshot from the ESP32. Requires header `X-Api-Key`. Returns `{status: ok, ts: ...}`.
GET	`/api/latest`	Returns the most recent snapshot as JSON.
GET	`/api/history?limit=N`	Returns the last N snapshots from SQLite, newest first (default N = 100, max 1000).
GET	`/api/cloud-log`	Returns the last 100 cloud communication events as a JSON array of timestamped strings.

10.6.1 Ingest Payload Schema

The JSON body sent by the ESP32 to `POST /api/ingest` contains the fields listed in Table 8. All fields are optional in the sense that the server accepts partial payloads, but the firmware always sends a complete snapshot.

10.6.2 Example API Interaction

The following `curl` command simulates an ESP32 posting a minimal snapshot to the server during testing:

Table 8: ESP32 JSON Payload Fields

Field	Type	Description
`connected`	bool	Always `true` when sent by the ESP32
`source`	string	Always `"esp32-sim800l"`
`device_id`	string	Configurable in `secrets.h`
`timestamp`	string	Network time from `AT+CCLK?`
`frame_count`	int	Cumulative CAN frames decoded since boot
`soc`	float	Voltage-derived SOC [%]
`soc_bms`	int	BMS coulomb-counter SOC [%]
`pack_v`	float	Pack voltage [V]
`pack_current_a`	float	Pack current [A], + = discharge, − = charge
`cell_count`	int	Number of non-zero cells decoded
`cell_avg_mv`	int	Average cell voltage [mV]
`cell_max_mv`	int	Maximum cell voltage [mV]
`cell_min_mv`	int	Minimum cell voltage [mV]
`cell_spread_mv`	int	Max − min cell voltage [mV]
`cells`	object	Per-cell `{"mv": int, "status": string}` map
`temps`	object	Per-probe temperature map [°C]
`avg_temp`	float	Average across active probes [°C]
`fault_level`	int	0 = normal, 1 = fault
`error_code`	int	BMS fault code (see Table 2)
`fault_name`	string	Human-readable fault description
`status`	object	Bit-field flags (charging, discharging, ready, etc.)
`charger`	object	Charging request limits (V, A, start signal)

```
curl -X POST http://YOUR_VPS_IP:8765/api/ingest \
  -H "Content-Type: application/json"           \
  -H "X-Api-Key: bako-bms-2024"                \
  -d '{
    "connected": true,
    "soc": 87.4,
    "pack_v": 62.31,
    "cell_count": 19
  }'
```

Response:

```
{"status": "ok", "ts": "2026-04-17T09:32:11.123456"}
```

The server then pushes this payload to any browser connected to `/ws/cloud` within 2 seconds.

11 INTEGRATION PHASE

11.1 Hardware-Software Integration (HSI)

11.2 Sub-system Integration Testing

11.3 Cloud-to-Vehicle Integration

11.4 Mechanical-Electrical Assembly Integration

11.5 Integration Test Report

12 IMPLEMENTATION PHASE

12.1 Prototype Build & Bring-Up

12.2 Validation & Verification (V&V)

12.3 Performance Testing

12.4 Regulatory & Homologation Testing

12.5 Production Readiness Review (PRR)

13 SCRUM & AGILE PROJECT MANAGEMENT

13.1 Agile Framework Overview

13.2 Sprint Log & Velocity Tracking

13.3 Product Backlog & Prioritization

13.4 Sprint Review & Retrospective Summaries

13.5 Definition of Done (DoD) & Acceptance Criteria

13.6 Burndown & Burnup Charts

14 BILL OF MATERIALS (BOM)

14.1 Electronics BOM

14.2 Mechanical & Structural BOM

14.3 Battery & Powertrain BOM

14.4 Software & Licensing BOM

14.5 BOM Cost Summary & Target vs. Actual

15 RESOURCES

15.1 Human Resources & Skillset Matrix

15.2 Equipment & Lab Resources

15.3 Software Tools & Licenses

15.4 Cloud & Infrastructure Resources

15.5 Budget Allocation & Burn Rate

16 LIMITATIONS

16.1 Technical Limitations

16.2 Regulatory & Certification Limitations

16.3 Budget & Resource Constraints

16.4 Timeline Limitations

16.5 Technology Readiness Level (TRL) Limitations

17 BLOCKAGES & ISSUE LOG

17.1 Active Blockers Register

17.2 Supply Chain & Procurement Blockages

17.3 Technical Deadlocks

17.4 Dependency & Third-Party Blockages

17.5 Resolved Blockers Log

18 RISK MANAGEMENT

18.1 Risk Register (Full)

18.2 Safety Risk Analysis (FMEA / HAZOP)

18.3 Cybersecurity Risk Assessment

18.4 Risk Review Cadence

19 QUALITY ASSURANCE

19.1 Quality Management Plan

19.2 Design Review Gates

19.3 Test & Measurement Plan

19.4 Defect Tracking & Resolution

20 RESULTS AND FINDINGS

20.1 Technical Performance Results

20.2 Software & Connectivity Results

20.3 Cost vs. Budget Analysis

20.4 Key Findings Summary

21 RECOMMENDATIONS

21.1 Technical Recommendations

21.2 Process Recommendations

21.3 Future Development Roadmap

22 CONCLUSION

REFERENCES

Generally, only references cited in the text are included in the references list; however, an occasional exception can be found to this rule. For example, supervisors may require evidence that students are familiar with a broader spectrum of literature than that immediately relevant to their study/report. In such instances, the reference list is called a bibliography. All material must be referenced using the IEEE style as indicated in the MedTech Internship guide. All sources must be acknowledged in your reports each time you use a finding from someone's work. For more details: https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

APPENDICES

A Glossary of Terms & Abbreviations

B Schematic & PCB Layout Package

C 3D Model & Drawing Package

D Software Repository & Build Instructions

E Test Reports & Certificates

F Supplier Datasheets & Approvals

G Meeting Minutes & Decision Log

H Revision History